Removing the Cast A Differential-Testing Tutorial

C-to-Common-Lisp quicksort

September 2026

The orthopedic metaphor

The C implementation is the Oracle: it preserves the behavior that the new Common Lisp implementation must match. CFFI is the temporary Cast. The Cast helps Lisp walk while the algorithm still depends on C, but it is not the final athlete. We remove it from the top down, one function at a time. After every removal, the C Oracle sorts the same deterministic corpus as the partial Lisp implementation. Only matching results allow the next step.

This is differential testing, not a proof. It is strong regression evidence because the Oracle and the implementation under test execute independently and their structured integer vectors are compared directly.

Files and reproducibility

- `quick-sort-array-oracle.c`: untouched copy of the original C program.

- `oracle-cli.c`: command-line adapter around that preserved Oracle.
- `quick-sort-array.c`: the four-function C Cast, with `main` removed.
- `cast.lisp`: CFFI declarations and the staged Lisp replacements.
- `quick-sort-array.lisp`: final standalone Common Lisp program.
- `differential-tests.lisp`: deterministic Oracle comparisons and invariants.
- `run-stages.sh`: builds both C artifacts and verifies stages 0 through 4.

Run `./run-stages.sh`. Stage 0 checks the complete C Cast. Stages 1–4 then remove one dependency at a time. Every stage checks empty, singleton, duplicate, negative, sorted, reverse-sorted, and generated arrays.

Stage 0: fit the complete C Cast

CFFI loads `libquicksort-cast.dylib`. The initial public Lisp entry point delegates to C, so the baseline verifies the foreign-memory conversion, calling convention, and Oracle harness before any algorithm is translated.

```
(defun cast-quicksort-indexed (data size)
  (%c-quicksort-indexed data size)
  (values))
```

Verification: Stage 0 passed against the C Oracle.

Stage 1: replace `quicksortIndexed`

The outer function is the safest first removal. Lisp handles the empty-array guard and still leans on the C range sorter.

```
(defun cast-quicksort-indexed (data size)
  (unless (zerop size)
    (cast-quicksort-indexed-range data 0 (1- size)))
  (values))
```

Verification: Stage 1 passed against the C Oracle.

Stage 2: replace the range sorter

Lisp now owns the divide-and-conquer recursion. Partitioning remains in C. The guard on a zero pivot index exactly preserves the source program's control flow, while a high value below low terminates the right recursion.

```
(defun cast-quicksort-indexed-range (data low high)
  (when (< low high)
    (let ((pivot-index
           (cast-partition-indexed data high low low)))
      (unless (zerop pivot-index)
        (cast-quicksort-indexed-range
          data low (1- pivot-index)))
      (cast-quicksort-indexed-range
        data (1+ pivot-index) high)))
  (values))
```

Verification: Stage 2 passed against the C Oracle.

Stage 3: replace the partitioner

The recursive partition walk moves into Lisp. CFFI's `mem-aref` reads the same contiguous C integer buffer. Swapping is still delegated to the last piece of the C Cast.

```
(defun cast-partition-indexed (data high scan boundary)
  (if (< scan high)
      (let ((pivot (cffi:mem-aref data :int high))
            (current (cffi:mem-aref data :int scan)))
        (if (< current pivot)
            (progn
              (cast-swap-at data scan boundary)
              (cast-partition-indexed
                data high (1+ scan) (1+ boundary)))
            (cast-partition-indexed
              data high (1+ scan) boundary)))
      (progn
        (cast-swap-at data boundary high)
        boundary)))
```

Verification: Stage 3 passed against the C Oracle.

Stage 4: replace `swapAt`

The final C operation is removed. Lisp reads both cells before writing either, which retains the original swap behavior even when both indexes are equal.

```
(defun cast-swap-at (data i j)
  (let ((left (cffi:mem-aref data :int i))
        (right (cffi:mem-aref data :int j)))
    (setf (cffi:mem-aref data :int i) right
          (cffi:mem-aref data :int j) left))
  (values))
```

Verification: Stage 4 passed against the C Oracle.

The Cast is off

At Stage 4 every quicksort function is implemented in Common Lisp. The staged adapter still stores values in foreign memory so all five stages share an identical test boundary. We now remove that last piece of scaffolding by using a Lisp vector directly. The standalone program has no CFFI, shared-library, C, Quicklisp, or other external runtime dependency. Lisp can now play soccer.

```
(defun swap-at (data i j)
  (rotatef (aref data i) (aref data j))
  (values))

(defun partition-indexed (data high scan boundary)
  (if (< scan high)
      (if (< (aref data scan) (aref data high))
          (progn
            (swap-at data scan boundary)
            (partition-indexed
              data high (1+ scan) (1+ boundary)))
          (partition-indexed data high (1+ scan) boundary))
      (progn (swap-at data boundary high) boundary)))

(defun quicksort-indexed-range (data low high)
  (when (< low high)
    (let ((pivot-index
            (partition-indexed data high low low)))
      (unless (zerop pivot-index)
        (quicksort-indexed-range data low (1- pivot-index)))
      (quicksort-indexed-range data (1+ pivot-index) high)))
  (values))

(defun quicksort-indexed (data size)
  (unless (zerop size)
    (quicksort-indexed-range data 0 (1- size)))
  data)
```

Run the original example or supply integers on the command line:

```
sbcl --script quick-sort-array.lisp
sbcl --script quick-sort-array.lisp 9 -2 7 7 0
```

The final dependency-free suite compares 72 deterministic vectors with the C Oracle and directly exercises all four Lisp functions and the command-line entry point. Run the complete history, including this final suite, with `./run-stages.sh`.